Laborator Python: funcții, metode și importuri pe web

Laborator pentru studenții din anul I

Nume și prenume: Stratulat Bogdan

Grupa: SI-262 Data: 29.09.2026

Prezentarea laboratorului

În acest laborator veți scrie programe Python care comunică cu un site web real și veți exersa trei concepte de bază: **importurile**, **metodele** și **funcțiile**. La final veți avea propriul modul Python, care generează un raport despre un site web.

După acest laborator veți putea:

- Să importați module din biblioteca standard și module externe și să explicați diferența dintre ele.
- Să apelați metode și să citiți atribute ale obiectelor create de alt cod (răspunsuri, șiruri de caractere, dicționare).
- Să scrieți propriile funcții, cu parametri, valori implicite, valori returnate, docstring-uri și tratarea erorilor.
- Să împărțiți codul într-un modul propriu și să îl importați dintr-un alt fișier.

Ce trebuie să știți înainte: variabile, instrucțiuni `if`, bucle `for`, liste și dicționare.

Site-ul țintă: toate exercițiile de citire folosesc `https://cybercor.org`. Exercițiile care *trimit* date folosesc `https://httpbin.org`, un serviciu public creat pentru testarea cererilor.

Termeni web esențiali

Termen	Ce înseamnă	Exemplu
URL	Adresa unei resurse pe web	`https://cybercor.org/robots.txt`
Cerere (request)	Mesajul pe care programul vostru îl trimite unui server	`GET /`
Răspuns (response)	Răspunsul serverului la cerere	cod de stare + antete + corp
Cod de stare (status code)	Un număr care arată cum s-a încheiat cererea	`200` OK, `301` mutat, `404` negăsit
Antet (header)	O pereche nume–valoare cu informații suplimentare	`Content-Type: text/html`
Corp (body)	Conținutul propriu-zis returnat	codul HTML al paginii
HTML	Limbajul de marcare în care sunt scrise paginile web	`<title>CYBERCOR</title>`

Importuri, metode și funcții, pe scurt

- Un **import** aduce în programul vostru cod scris de altcineva: `import requests`.

- O **metodă** este o funcție care aparține unui obiect și se apelează cu punct: `response.json()`, `text.lower()`.
- O **funcție** este un bloc de cod cu nume, reutilizabil, pe care îl scrieți voi: `def get_title(html): ...`.

Configurare și reguli

Înainte de exercițiul 1, instalați o bibliotecă externă și creați un director pentru laborator.

1. Verificați versiunea Python cu `python --version` (3.9 sau mai nouă).
2. Instalați biblioteca `requests`: `pip install requests`.
3. Creați un director numit `web_lab` și lucrați în el. Salvați fiecare parte într-un fișier separat: `part1_imports.py`, `part2_methods.py` și așa mai departe.
4. Începeți fiecare fișier cu acest șablon:

```
# Laborator: funcții, metode și importuri pe web
# Student: <numele vostru>

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

Reguli de lucru

Lucrați cu un site web real, de care depind oameni reali. Respectați aceste reguli la fiecare exercițiu:

- **Folosiți mereu un timeout.** Fiecare cerere primește `timeout=TIMEOUT`, ca programul să nu se blocheze la nesfârșit.
- **Faceți pauze între cereri.** Când trimiteți mai multe cereri una după alta, puneți `time.sleep(1)` între ele.
- **Doar citire.** Către cybercor.org trimiteți doar cereri `GET` și `HEAD`. Tot ce trimite date (`POST`, parametri de interogare, antete personalizate pe care vreți să le verificați) merge către httpbin.org.
- **Nicio buclă fără limită.** Nu scrieți niciodată o buclă `while True:` care trimite cereri.
- **Rămâneți pe țintă.** Accesați doar URL-urile indicate în exerciții. Nu scanați alte site-uri.

Încălcarea acestor reguli poate duce la blocarea adresei IP a întregii săli, de aceea respectarea lor face parte din notă.

Partea 1: Importuri (exercițiile 1–8)

Scop: învățați diferitele moduri de a aduce cod în program. Fișier: `part1_imports.py`.

Exercițiul 1. **Primul import.** Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

Exercițiul 2. **Două stiluri de import.** Descărcați pagina `BASE_URL` cu `requests.get()` și afișați codul de stare. Apoi rescrieți același script folosind `from requests import get`. Într-un comentariu, scrieți câte un avantaj al fiecărui stil.

Exercițiul 3. Alias-uri. Importați requests cu un alias: `import requests as rq`. Descărcați pagina principală cu `rq.get()`. Într-un comentariu, spuneți când un alias face codul mai ușor de citit și când îl face mai greu de citit.

Exercițiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

Exercițiul 5. Priviți în interiorul unui modul. Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

Exercițiul 6. Citiți documentația. Rulați `help(requests.get)`. Găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

Exercițiul 7. Cronometrarea unei cereri. Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

Exercițiul 8. Când un import eșuează. Scrieți un script care încearcă `import bs4` într-un bloc `try` / `except ImportError`. Dacă modulul lipsește, afișați: `Instalați modulul cu: pip install beautifulsoup4`.

> **Verificați-vă:** care este diferența dintre un modul, un pachet și o bibliotecă?

Partea 2: Metode și atribute ale obiectelor web (exercițiile 9–20)

Scop: învățați să folosiți instrumentele pe care obiectele le au deja. Un **atribut** este o valoare (`response.status_code`); o **metodă** este o acțiune pe care o apelați cu paranteze (`response.json()`). Fișier: `part2_methods.py`.

Exercițiul 9. Elementele de bază ale răspunsului. Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

Exercițiul 10. Tratarea erorilor cu o metodă. Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

Exercițiul 11. Toate antetele. Parcurgeți `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

Exercițiul 12. Metode de dicționar. Folosiți `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipsește"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

Exercițiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

Exercițiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

> **Indiciu:** `start = html.find("<title>") + len("<title>")`

Exercițiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

Exercițiul 16. Verificarea HTTPS. Folosiți `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

Exercițiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

Exercițiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

Exercițiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

Exercițiul 20. Sesiuni. Creați un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-<numele vostru>"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.

> **Verificați-vă:** atât `response.text`, cât și `response.json()` vă dau corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

Partea 3: Scrierea propriilor funcții (exercițiile 21–34)

Scop: organizați codul în funcții reutilizabile. De acum înainte, fiecare exercițiu este o funcție pe care o apelați cel puțin o dată. Fișier: `part3_functions.py`.

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code
```

Exercițiul 21. Prima funcție. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

Exercițiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

Exercițiul 23. Parametri impliciți. Modificați funcția în `fetch(url, timeout=10)`. Apelați-o o dată cu valoarea implicită și o dată cu `timeout=3`.

Exercițiul 24. O singură sarcină pe funcție. Scrieți `get_title(html)`, care primește un șir HTML și returnează titlul paginii, refolosind logica din exercițiul 14. Observați că această funcție nu face nicio operație de rețea: descărcarea și analiza sunt acum separate.

Exercițiul 25. Docstring-uri. Adăugați un docstring funcției `get_title` și afișați-l cu `help(get_title)`.

Exercițiul 26. Adnotări de tip. Adăugați adnotări de tip (type hints) funcțiilor, de exemplu `def get_status(url: str) -> int:`. Apelați `get_status(123)` și observați ce se întâmplă. Vă opresc adnotările de tip?

Exercițiul 27. Fără opriri neașteptate. Scrieți `page_exists(url)`, care returnează `True` sau `False`. Folosiți `try` / `except requests.RequestException`, astfel încât funcția să returneze `False` în loc să se oprească cu eroare, chiar și pentru un URL precum `https://this-domain-does-not-exist.invalid`.

Exercițiul 28. Returnarea unui dicționar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

Exercițiul 29. Argumente cu nume. Scrieți `get_header(url, name, default="lipsește")`, care returnează valoarea unui singur antet. Apelați-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

Exercițiul 30. Antete de securitate. Scrieți `security_headers(url)`, care verifică prezența acestor cinci antete și returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

Exercițiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

Exercițiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

Exercițiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dicționar `{url: secunde}`. Apelați-o cu două URL-uri.

Exercițiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.

> **Verificați-vă:** care este diferența dintre a afișa o valoare cu `print()` în interiorul unei funcții și a o returna cu `return`?

Partea 4: Alte module din biblioteca standard (exercițiile 35–43)

Scop: combinați mai multe importuri pentru a rezolva probleme mai mari. Puneți fiecare soluție într-o funcție. Fișier: `part4_modules.py`.

Modul	La ce îl folosiți aici
`urllib.parse`	Descompunerea și compunerea URL-urilor
`re`	Căutarea de tipare în text
`html.parser`	Citirea corectă a codului HTML
`hashlib`	Calcularea amprentei unei pagini
`json`	Salvarea datelor într-un fișier
`socket`	Aflarea adreselor IP și deschiderea conexiunilor

Modul	La ce îl folosiți aici
`ssl` + `datetime`	Citirea datei de expirare a unui certificat

Exercițiul 35. Descompunerea unui URL. Folosiți `urllib.parse.urlparse()` pe `https://cybercor.org/path?x=1#top`. Afișați `.scheme`, `.netloc`, `.path`, `.query` și `.fragment`.

Exercițiul 36. Compunerea URL-urilor. Folosiți `urllib.parse.urljoin()` pentru a transforma legăturile relative `/about`, `contact.html` și `../index.html` în URL-uri complete, pornind de la `BASE_URL`.

Exercițiul 37. Extragerea legăturilor. Folosiți `re.findall(r'href="([^"]+)"', html)` pentru a obține toate legăturile de pe pagina principală. Puneți codul într-o funcție `extract_links(html)`, care returnează legăturile fără duplicate.

Exercițiul 38. Interne sau externe? Scrieți `split_links(links, domain)`, care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri. Folosiți mai întâi `urljoin()`, apoi `urlparse().netloc`.

Exercițiul 39. Propriile metode. Creați o clasă care moștenește `html.parser.HTMLParser` și suprascrieți metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

> **Cum testați exercițiul 39.** Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                          # imagine obișnuită
    "poza.jpg",                           # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

Exercițiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

Exercițiul 41. Salvarea în JSON. Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

Exercițiul 42. Interogare DNS. Scrieți `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afișați adresa IP a domeniului `cybercor.org`.

Exercițiul 43. Expirarea certificatului. Scrieți `cert_days_left(hostname)`, care se conectează la portul 443, citește certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiți data cu `ssl.cert_time_to_seconds()`.

> **Verificați-vă:** în exercițiul 39, cine apelează `handle_starttag()`? Voi nu o apelați niciodată direct.

Partea 5: Construirea propriului modul (exercițiile 44–50)

Scop: transformați funcțiile într-un modul pe care alte programe îl pot importa, la fel ca `requests`. La final, directorul vostru ar trebui să arate astfel:

```
web_lab/
├── webtools.py     # modulul vostru: toate funcțiile reutilizabile
├── main.py         # programul care importă webtools
├── report.csv      # creat în exercițiul 49
└── report.json     # creat în exercițiul 50
```

Exercițiul 44. Crearea unui modul. Mutați `fetch`, `get_status`, `get_title` și `security_headers` într-un fișier nou, `webtools.py`. În `main.py`, scrieți `import webtools` și apelați `webtools.get_title(webtools.fetch(BASE_URL).text)`.

Exercițiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicați ce s-ar întâmpla dacă `main.py` ar avea și o funcție proprie numită `get_title`.

Exercițiul 46. Blocul main. Adăugați acest bloc la finalul fișierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulați `python webtools.py`, apoi `python main.py`. Explicați de ce autotestul rulează doar în primul caz.

Exercițiul 47. Constante de modul. Adăugați constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-<numele vostru>"}` în `webtools.py` și folosiți-o în `fetch()`. În `main.py`, importați constanta și afișați-o.

Exercițiul 48. Argumente din linia de comandă. Folosiți `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteți argumentul URL funcțiilor voastre, în loc să folosiți `BASE_URL`.

Exercițiul 49. Un raport CSV. Folosiți modulul `csv` pentru a salva rezultatul funcției `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folosiți `datetime.now().isoformat()`.

Exercițiul 50. Proiect final: raportul site-ului. Scrieți `site_report(url)` în `webtools.py`. Funcția trebuie să folosească funcțiile voastre pentru a colecta toate informațiile de mai jos, să le afișeze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lanțul de redirecționări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului

7. Numărul de legături interne și externe
8. Căile interzise din `robots.txt`

Un exemplu de rezultat (valorile voastre vor fi diferite):

```
=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         203.0.113.10
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        74 de zile rămase
Legături:          38 interne, 12 externe
Căi interzise:     /admin, /private
Salvat în report.json
```

> **Bonus:** instalați `beautifulsoup4` și rescrieți `get_title()` și `extract_links()` folosind această bibliotecă. Comparați cât de mult mai scurte și mai sigure devin.

Predare și notare

Predați întregul director `web_lab` ca o singură arhivă `.zip`, numită `web_lab_<numele_vostru>.zip`.

Înainte de predare

- ☐ Fiecare fișier rulează fără erori de la început până la sfârșit.
- ☐ Fiecare cerere folosește un timeout, iar între cererile repetate există `time.sleep(1)`.
- ☐ Fiecare funcție, începând cu Partea 3, are un docstring.
- ☐ Toate cerințele de tip "explicați într-un comentariu" au răspuns în cod.
- ☐ `python main.py https://cybercor.org` afișează raportul complet și creează `report.json`.
- ☐ Directorul conține `report.csv` și `report.json`.

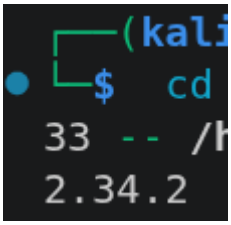
Notare

Partea	Exerciții	Puncte
Partea 1: Importuri	1–8	10
Partea 2: Metode și atribute	9–20	20
Partea 3: Propriile funcții	21–34	25
Partea 4: Module din biblioteca standard	35–43	20
Partea 5: Propriul modul și proiectul final	44–50	20
Respectarea regulilor de lucru	toate	5
Total		**100**

Exercițiul bonus (BeautifulSoup): până la 5 puncte în plus.

Exercitiu 1

```python
#Exercitiu 1
import requests
print(requests.__version__)
import urllib.request
# Modulul requests nu face parte din biblioteca standard Python, de aceea trebuie instalat separat cu comanda: pip install requests.
# Modulul urllib face parte din biblioteca standard Python, deci este instalat automat împreună cu Python și nu necesită pip install.
```



Exercitiu 5

```python
#Exercitiu 5
import requests
print(dir(requests))
# get - funcție
# Session - clasă
# exceptions - modul
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$ cd /home/kali/web_lab_stratulat_bogdan ; /usr/bin/env /bin/python /home/kali/.vscode-oss/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 34555 -- /home/kali/web_lab_stratulat_bogdan/part1_import.py
tsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequired', '__all__', '__author__', '__author_email__', '__build__', '__builtins__', '__cached__', '__cake__', '__copyright__', '__description__', '__doc__', '__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__', '__url__', '__version__', '_check_cryptography', '_internal_utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', 'charset_normalizer_version', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post', 'put', 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils', 'warnings']
```

Exercitiu 10

```python
#Exercitiu 10
import requests
response=requests.get(BASE_URL,timeout=TIMEOUT)
response.raise_for_status()
try:
    response=requests.get(BASE_URL + "/this-page-does-not-exist")
    response.raise_for_status()
    print("Pagina exista")
except requests.HTTPError:
    print("Pagina nu există sau a apărut o eroare HTTP.")
```

```
┌──(kali㉿kali
└─$  cd /home/
49 -- /home/ka
Pagina exista
```

Exercitiu 15

```python
#Exercitiu 15
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
lines = response.text.splitlines()
print("Nr. linii" ,len(lines))
cea_mai_lunga = max(lines, key=len)
print("Lungimea celei mai lungi linii:", len(cea_mai_lunga))
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_
└─$  cd /home/kali/web_lab_stratulat_
81 -- /home/kali/web_lab_stratulat_b
Nr. linii 32
Lungimea celei mai lungi linii: 218
```

Exercitiu 20

```python
#Exercitiul 20
import requests
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-Stratulat Bogdan"})
response = session.get(ECHO_URL + "/headers")
print(response.text)
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$  cd /home/kali/web_lab_stratulat_bogdan ; /usr/bin/env /bin/py
47 -- /home/kali/web_lab_stratulat_bogdan/part2_methods.py
    "User-Agent": "WebLab-Stratulat Bogdan",
    "X-Amzn-Trace-Id": "Root=1-6accbdc5-301c25602045e7de1807a273"
  }
}
```

Exercitiu 25

```python
#Exercitiu 25
def get_title(html):
    """Returnează titlul unei pagini HTML."""
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    return html[start:end].strip()
help(get_title)
```

```
   Returnează titlul unei pagini HTML.
Help on get_title line 1/4 (END) (press h for help or q to quit)

     Commands marked with * may be preceded by a number, N.
     Notes in parentheses indicate the behavior if N is given.
     A key preceded by a caret indicates the Ctrl key; thus ^K is ctrl-K.

  h  H                 Display this help.
  q  :q  Q  :Q  ZZ     Exit.
 ---------------------------------------------------------------------------
HELP -- Press RETURN for more, or q when done
```

Exercitiu 30

```python
#Exercitiu 30
def security_headers(url):
    response = requests.get(url, timeout=TIMEOUT)
    antete = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    rezultate = {}
    for antet in antete:
        rezultate[antet] = antet in response.headers
    return rezultate
print(security_headers(BASE_URL))
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$  cd /home/kali/web_lab_stratulat_bogdan ; /usr/bin/env /bin/python /home/kali/.vscode-oss/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/li
93 -- /home/kali/web_lab_stratulat_bogdan/part3_functions.py
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}
```

Exercitiu 35

```python
#Exercitiu 35
import urllib
response = urllib.parse.urlparse("https://cybercor.org/path?x=1#top")
print(response.scheme)
print(response.netloc)
print(response.path)
print(response.query)
print(response.fragment)
```

```
┌──(kali㉿kal
└─$  cd /home
27 -- /home/k
https
cybercor.org
/path
x=1
top
```

Exercitiu 40

```python
#Exercitiu 40
import hashlib
def page_fingerprint(url):
    response = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(response.content).hexdigest()
amprenta1 = page_fingerprint(BASE_URL)
amprenta2 = page_fingerprint(BASE_URL)
print(amprenta1)
print(amprenta2)
if amprenta1 == amprenta2:
    print("Amprentele sunt identice.")
else:
    print("Amprentele sunt diferite.")
# Amprentele vor fi diferite dacă se schimbă conținutul paginii între cele două cereri.
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$  cd /home/kali/web_lab_stratulat_bogdan ; /usr/bin/env /bin/p
07 -- /home/kali/web_lab_stratulat_bogdan/part4_modules.py
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
Amprentele sunt identice.
```

Exercitiu 45

```python
#Exercitiu 45
import requests
from webtools import get_title, security_headers
response = requests.get(BASE_URL, timeout=10)
print(get_title(response.text))
print(security_headers(BASE_URL))
# Dacă main.py ar avea și o funcție proprie numită get_title,
# una dintre ele ar suprascrie-o pe cealaltă, în funcție de ordinea definirii/importului.
```

```
┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$  cd /home/kali/web_lab_stratulat_bogdan ; /usr/bin/env /bin/python /home/kali/.vscode-oss/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/li
93 -- /home/kali/web_lab_stratulat_bogdan/main.py
CYBERCOR
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}
```

Exercitiu 50

```python
#Exercitiu 50
import socket
import json
import urllib.parse
from part4_modules import resolve, cert_days_left, extract_links, split_links
from part3_functions import score_headers, fetch_robots, disallowed_paths

def site_report(url):
    response = fetch(url)

    # 1. Cod de stare și URL final
    status = response.status_code
    final_url = response.url

    # 2. Titlu
    titlu = get_title(response.text)

    # Domeniul final
    hostname = urllib.parse.urlparse(final_url).hostname
    domain = urllib.parse.urlparse(final_url).netloc

    # 3. Adresa IP
    ip = resolve(hostname)

    # 4. Redirecționări pornind de la http://
    http_url = final_url.replace("https://", "http://", 1)
    redirect_response = fetch(http_url)

    redirects = []

    for r in redirect_response.history:
        locatie = r.headers.get("Location", "")
        redirects.append(str(r.status_code) + " -> " + locatie)

    # 5. Scor securitate
    securitate = security_headers(final_url)
    scor = score_headers(securitate)

    # 6. Certificat
    zile = cert_days_left(hostname)

    # 7. Legături interne și externe
    links = extract_links(response.text)
    interne, externe = split_links(links, domain)

    # 8. robots.txt
    robots = fetch_robots(final_url)
    cai = disallowed_paths(robots)

    raport = {
        "status": status,
        "url_final": final_url,
        "titlu": titlu,
        "ip": ip,
        "redirectionari": redirects,
        "scor_securitate": scor,
        "certificat_zile": zile,
        "legaturi_interne": len(interne),
        "legaturi_externe": len(externe),
        "cai_interzise": cai
    }

    print("=== Raport site:", final_url, "===")
    print("Cod de stare:", status)
    print("URL final:", final_url)
    print("Titlu:", titlu)
    print("Adresă IP:", ip)
    print("Redirecționări:", redirects)
    print("Scor securitate:", scor)
    print("Certificat:", zile, "de zile rămase")
    print("Legături:", len(interne), "interne,", len(externe), "externe")
    print("Căi interzise:", cai)

    with open("report.json", "w") as file:
        json.dump(raport, file, indent=2)

    print("Salvat în report.json")

    return raport
```

```
55  #Exercitiu 50
56  from webtools import site_report
57  site_report(BASE_URL)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

┌──(kali㉿kali)-[~/web_lab_stratulat_bogdan]
└─$  /usr/bin/env /bin/python /home/kali/.vscode-
/main.py
=== Raport site: https://cybercor.org/ ===
Cod de stare: 200
URL final: https://cybercor.org/
Titlu: CYBERCOR
Adresă IP: 172.67.211.96
Redirecționări: ['301 -> https://cybercor.org/']
Scor securitate: 3/5
Certificat: 52 de zile rămase
Legături: 1 interne, 0 externe
Căi interzise: []
Salvat în report.json
```

Concluzie

În concluzie, în urma acestui laborator am învățat mai bine cum se folosesc funcțiile, modulele și bibliotecile în Python. Am lucrat cu requests și am văzut cum putem obține diferite informații despre un site, cum ar fi codul de stare, antetele, linkurile, cookie-urile sau redirecționările. De asemenea, am folosit fișiere JSON și CSV, expresii regulate, URL-uri, DNS și certificate SSL.

Pe parcurs am înțeles mai bine și de ce este util să împărțim codul în funcții și module separate, pentru ca programul să fie mai ordonat și mai ușor de refolosit. La final, am adunat funcțiile într-un modul propriu, webtools.py, și le-am folosit din alt fișier. Per total, laboratorul m-a ajutat să înțeleg mai bine lucrul cu pagini web în Python și modul în care poate fi organizat un program mai mare.